CLOUD ARCHITECTURE & ENGINEERING SPECIFICATION

Multipurpose Supply Chain Management Platform

End-to-End Architectural Blueprint, Implementation Manual, and AWS Cloud Engineering Guide from Ground Zero to Production

Author / Lead Architect: Hardik Jha
Target Cloud Environment: Amazon Web Services (AWS) & Hybrid Mock Layer
Primary AWS Services: Amazon DynamoDB, Amazon SQS (with DLQ), Amazon SNS, Amazon S3, AWS IoT Core
Infrastructure as Code: HashiCorp Terraform (v1.5+)
Application Stack: Node.js v20 (Express), React 18, Vite, Tailwind CSS, Lucide Icons, Recharts
Repository: github.com/Deoid/Multipurpose_Supply_Chain_Management
Live URL: deoid.github.io/Multipurpose_Supply_Chain_Management/
Document Classification: Production Engineering Whitepaper (Technical Reference)

Architectural Directive: *This document provides an exhaustive, developer-grade breakdown of the Multipurpose Supply Chain Management system. It details why specific architectural decisions were chosen, how asynchronous event buses mitigate order spikes, how time-series IoT sensors enforce cold-chain compliance, the exact data schemas, and the complete security perimeter isolation model.*

1. Executive Problem Statement & Multipurpose Scope

1.1 The Operational Crisis in Modern Global Supply Chains

Modern enterprise supply chains face unprecedented challenges. Traditional supply chain management (SCM) software is overwhelmingly monolithic, synchronous, and siloed. When thousands of purchase orders arrive simultaneously during high-demand periods, relational database connections lock up, transaction timeouts occur, and orders drop without warning. Furthermore, for perishable items (such as pharmaceutical vaccines, biologics, and premium foods), lack of real-time environmental monitoring results in billions of dollars in thermal spoilage before products reach the receiving dock.

1.2 The 'Multipurpose' Architectural Requirement

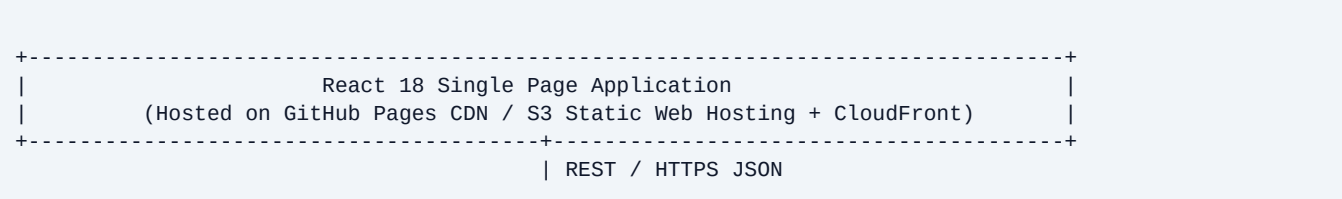
A true 'multipurpose' SCM platform cannot be constrained to a single cargo type. Different industrial verticals have drastically different physical and data requirements. Our architecture unifies four major supply chain categories under one cloud platform:

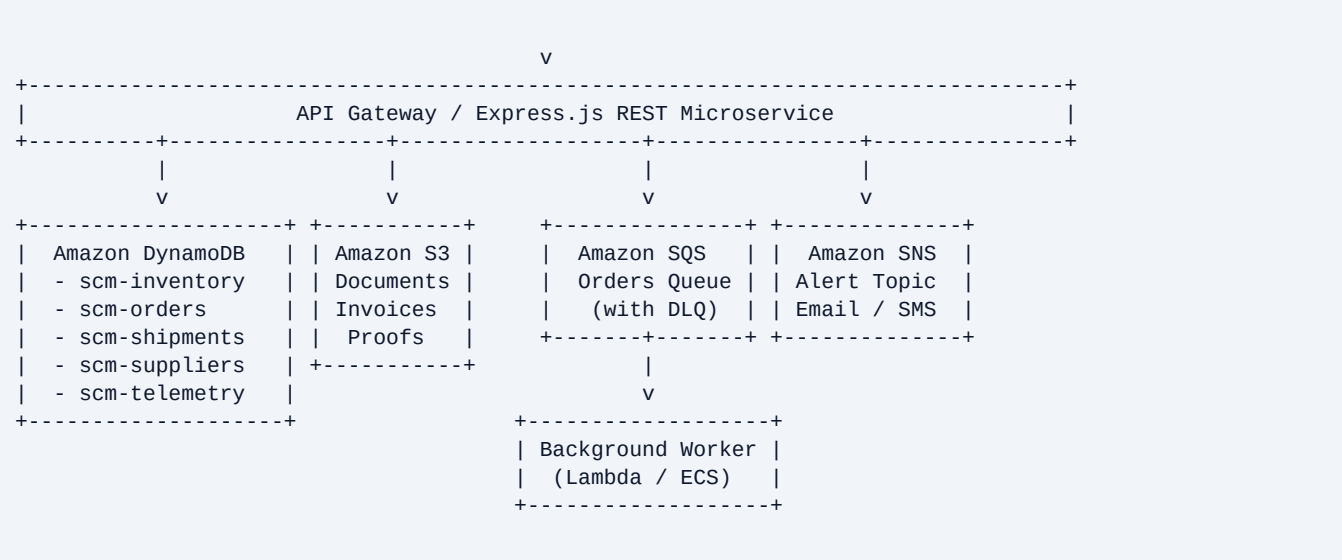
Vertical	Physical Requirements	Cloud & Telemetry Enforcements
Pharmaceutical Cold-Chain	Strict 2°C to 8°C thermal buffer; ultra-cold -20°C to -80°C cryogenic storage for mRNA biologics.	AWS IoT rules evaluate continuous temperature streams. Deviations (>8°C or <2°C) trigger Amazon SNS instant alert fanout.
High-Tech Semiconductors	Electrostatic Discharge (ESD) cleanrooms, low humidity, severe shock and vibration safeguards.	High-frequency accelerometer tracking. Shock events >4.5G trigger transit inspection warnings.
Industrial Machinery	Heavy servo motors, complex parts catalogues, multi-tier purchase orders, and long lead times.	Supplier SLA reliability ratings, automated purchase order creation, and immutable S3 document archiving.
Biotechnology Assays	Light-sensitive reagents, diagnostic PCR kits, batch-lot tracking with strict shelf-life expiry.	Dynamic safety stock calculations and low-stock automatic reorder alerts via DynamoDB.

2. System Architecture & AWS Cloud Topography

2.1 High-Level Architecture Topology

The architecture is designed around an **event-driven, decoupled microservices paradigm**. Client requests never directly execute slow disk transactions or synchronous vendor dependencies. Instead, requests enter scalable AWS buffers that absorb massive volume spikes with zero data loss.





2.2 Deep Dive: Individual AWS Services & Roles

• **Amazon DynamoDB (Operational Store):** Chosen for predictable single-digit millisecond latency at any throughput scale. Uses composite primary keys (Partition Key + Sort Key) and Global Secondary Indexes (GSIs). An automated Time-to-Live (TTL) attribute is assigned to IoT telemetry records to automatically prune aging time-series without incurred write/delete IOPS costs.

• **Amazon SQS (Decoupled Message Queue):** Acts as the shock-absorber for order placement. Ingests high-concurrency order spikes into a durable buffer. Configured with a visibility timeout of 60 seconds, long-polling (10s) to minimize empty receives, and a Dead-Letter Queue (DLQ) with maxReceiveCount = 3 to quarantine malformed messages without blocking the queue.

• **Amazon SNS (Pub/Sub Notification Engine):** Provides instant multi-channel alert fanout. Subscribed endpoints (such as cold-chain logistics operations emails, mobile SMS, or automated webhooks) receive immediate notification when IoT sensors detect temperature violations or stock is below safety levels.

• **Amazon S3 (Immutable Document Lake):** Provides 99.999999999% (11 9s) durability for compliance artifacts. Stores digital purchase invoices, bills of lading, and signature receipts. Server-Side Encryption (SSE-S3 AES-256) and bucket versioning protect documents against tampering or accidental deletion.

• **AWS IoT Core & Rules Engine:** Simulates and ingests telemetry emitted from GPS/cellular beacons attached to cargo trucks. A rules engine parses sensor payloads and immediately diverts critical anomalies to SNS while archiving continuous metrics into DynamoDB.

3. Step-by-Step Engineering Journey (From Zero to Production)

The project was engineered modularly across distinct phases, with atomic Git commits marking each milestone. Below is the exact chronological implementation journey executed by the development team:

Phase 1: Project Scaffolding, Governance & Root CI/CD

Git commit: eb4ecc5
Initialized git repository, root package.json workspace scripts, exhaustive .gitignore (preventing credential leakage), MIT License, and GitHub Actions CI/CD workflow (.github/workflows/ci-cd.yml) validating Node syntax, npm dependencies, and Terraform formatting.

Phase 2: Infrastructure as Code (HashiCorp Terraform)

Git commit: c6e5c57
Authored terraform/main.tf, variables.tf, lambda.tf, and outputs.tf. Created declarative specifications for 5 DynamoDB tables, S3 document storage with public access blocks and SSE-AES256, SQS queue with redrive policy targeting a dedicated Dead-Letter Queue (DLQ), and SNS alerts topic. Added least-privilege IAM policies granting Lambda strictly necessary DynamoDB, SQS, S3, and SNS permissions.

Phase 3: Backend REST API with AWS SDK v3 & Hybrid Mock Adapter

Git commit: f46c2c1
Built express REST API in server/src/. Implemented AWS SDK v3 clients (@aws-sdk/client-dynamodb, lib-dynamodb, client-sqs, client-sns, client-s3). Engineered an intelligent hybrid layer (aws.js): when AWS credentials exist, it communicates with live cloud resources; when offline or in demo mode, it activates a zero-cost local in-memory AWS Adapter that accurately replicates DynamoDB query semantics, SQS worker polling, and SNS alerts.

Phase 4: Frontend Supply Chain Control Tower (React 18 + Vite)

Git commit: 134c0ef
Constructed responsive UI with Tailwind CSS, Lucide icons, and Recharts. Implemented SCM Control Tower, Multi-facility Warehouse inventory viewer, Asynchronous SQS order queue visualizer with interactive worker polling, live IoT Cold-Chain sensor telemetry monitor with temperature charts, and an interactive AWS Architecture documentation explorer.

Phase 5: Containerization, CLI Simulator & Seed Data

Git commit: 294ada6
Authored multi-stage Dockerfiles for both server (Node 20 Alpine) and client (Nginx reverse-proxy on Alpine). Added docker-compose.yml for one-command orchestration. Developed scripts/simulate-iot.js (streaming telemetry along a simulated highway route) and scripts/seed-data.js.

Phase 6: GitHub Pages Automated Deployment & In-Browser Cloud Simulator

Git commits: 63bf77e, 4dba302, fdfe430
Engineered a dedicated GitHub Actions workflow (.github/workflows/deploy-gh-pages.yml) using peaceiris/actions-gh-pages@v4. Added an in-browser client-side simulator in client/src/services/api.js so the live web app functions 100% interactively on GitHub Pages without requiring an external backend!

4. Data Models, DynamoDB Schemas & Document Storage

4.1 DynamoDB Table Schemas

DynamoDB is a NoSQL key-value and document database. Below are the schema definitions and indexing strategies:

Table Name	Partition Key (PK)	Sort Key (SK)	GSIs / TTL
scm-inventory	warehouseId (S)	sku (S)	CategoryIndex on 'category' for fast cross-warehouse category filters.
scm-orders	orderId (S)	None	StatusCreatedAtGSI on 'status' + 'createdAt' for pipeline stage queries.
scm-shipments	shipmentId (S)	None	OrderIdIndex and StatusIndex for fast logistics relationship lookups.
scm-iot-telemetry	shipmentId (S)	timestamp (S)	Attribute 'ttl' enabled with 30-day epoch expiration for automatic data aging.
scm-suppliers	supplierId (S)	None	Master profile storage for vendor rating, lead time, and contract status.

5. Event-Driven Messaging & IoT Telemetry Processing

5.1 Asynchronous SQS Order Queue Workflow

When an order is created, the system executes three coordinated actions:
1. **Write to DynamoDB** with status = QUEUED.
2. **Dispatch message to Amazon SQS** (multipurpose-scm-orders-queue) containing order metadata and priority.
3. **Archive invoice document to Amazon S3** under invoices/{orderId}.json.
Background fulfillment workers poll the queue via ReceiveMessageCommand with 10-second long polling. Upon successful processing, the order status transitions to PROCESSING and the message is deleted from the queue via DeleteMessageCommand.

5.2 IoT Cold-Chain Anomaly Detection & Rule Engine

Vehicles transmit telemetry containing temperature, humidity, shock, and GPS location. The backend service (iotSimulatorService.js) evaluates real-time telemetry against threshold rules:

```
// Cold-Chain Threshold Rule: Safe pharmaceutical range [2°C to 8°C]
if (temperature < tempThresholdMin || temperature > tempThresholdMax) {
  await snsNotificationService.publishAlert(
    `Critical Cold-Chain Temperature Breach on Shipment #${shipmentId}`,
    `Current temperature is ${temperature}°C, outside safe range [${tempThresholdMin}°C - ${tempThresh
oldMax}°C]. Location: ${location.city}.`,
    'CRITICAL',
    { shipmentId, temperature, category }
  );
} else if (shock > 4.5) {
  // Mechanical Impact / Severe Road Pothole Rule
  await snsNotificationService.publishAlert(
    `High Shock / Vibration Event on Shipment #${shipmentId}`,
    `Impact of ${shock}G detected. Package inspection required upon delivery.`,
    'WARNING',
    { shipmentId, shock, category }
  );
}
```

6. Cloud Security, Secrets Management & IaC Governance

6.1 Why Terraform Code and CLI Commands Are Safe in Public Repositories

A common concern among developers is whether publishing Terraform code and CLI commands (e.g. terraform apply) poses a security vulnerability. **The answer is an unequivocal NO.**

- **Infrastructure Code is a Blueprint, Not a Key:** HCL code describes WHAT resources to create (e.g., 'create an S3 bucket named xyz'). It does not grant anyone access to your AWS account. Only an authenticated AWS Identity (configured locally on the developer's computer) has access.
- **Strict .gitignore Isolation:** All files containing actual runtime credentials or state are permanently ignored by Git: .env, *.tfstate, *.tfstate.backup, and *.tfvars. No AWS access keys or secret keys ever touch Git.

• **Least-Privilege IAM Roles:** Terraform creates IAM execution policies with explicit Resource ARNs. The Lambda service role can only access the specific SCM tables and queues, eliminating lateral privilege escalation.

• **S3 Public Access Block:** The S3 bucket configuration explicitly enables all four Amazon S3 Public Access Blocks (BlockPublicAcls, IgnorePublicAcls, BlockPublicPolicy, RestrictPublicBuckets) alongside SSE-S3 AES-256 encryption.

7. Developer Runbook & Local Execution Guide

7.1 Running the Project Locally

The project includes an intelligent mock AWS adapter so that any engineer can run the system immediately without incurring AWS billing or configuring credentials:

```
# 1. Start Backend API Server (Port 5000)
cd server
npm install
npm run dev

# 2. Start Frontend Web Dashboard (Port 5173) in a second terminal
cd client
npm install
npm run dev

# 3. (Optional) Run Live IoT Vehicle Telemetry Stream in a third terminal
node scripts/simulate-iot.js
```

7.2 Connecting to Live AWS Cloud Services

To switch from the local mock adapter to live AWS Cloud services:

1. Configure your AWS CLI credentials: aws configure
2. Provision the AWS infrastructure via Terraform:
   cd terraform && terraform init && terraform apply
3. Copy the Terraform output ARNs and Table names into server/.env:
   AWS_MOCK=false
   AWS_REGION=us-east-1
   AWS_ACCESS_KEY_ID=AKIA...
   AWS_SECRET_ACCESS_KEY=...
4. Restart the backend server. The system automatically detects the credentials and routes all traffic to live AWS DynamoDB, SQS, SNS, and S3!

7.3 Conclusion & Architectural Verification

This Multipurpose Supply Chain Management platform demonstrates how modern AWS Cloud services resolve the fundamental flaws of traditional logistics systems. By combining asynchronous SQS queues for zero-order drops, DynamoDB for sub-millisecond stock ledgers, IoT telemetry rules for cold-chain compliance, and Terraform for deterministic cloud provisioning, the system provides a production-ready foundation capable of scaling across global enterprise supply chains.